营销活动

优惠券 / 秒杀 / 抢红包——三个 Lua 套路对照与防黑产

gomall · 业务侧 deck

今日大纲

把三个场景拼到一张桌子上

优惠券：库存 + 限领二维原子

秒杀：滑动窗口 + 单独商品表

抢红包：LPOP 一次取一份

防黑产：分层与现状

拼团 + 满减：营销玩法的两个常见扩展

排行榜 + 直播打赏：高频写场景的两种姿态

红包深挖 + 风控决策树

大促经验 + 优惠生命周期 + 容量规划

把三个场景拼到一张桌子上

优惠券 / 秒杀 / 抢红包凭什么放一起讲

- 表面三类活动，骨架其实是同一套：
  - 入口流量打到一个共享资源（库存 / 商品份额 / 红包份数）。
  - 必须原子扣减，否则超发 = 直接资金损失。
  - 防刷需求强：羊毛党盯的就是这种” 低价 / 免费” 场景。
  - 抢到只是第一步，还要落库 + 入账，链路天然要 Saga 兜底。
- gomall 三个接口的代码其实长得像三胞胎：都是**中间件链 → Lua 一锤 → 事务落库 → outbox 投递**。
- 本 deck 把三段 Lua 摆在一起读，挑差异、说原因。

对应 *service/coupon.go, service/redpacket.go, service/skill_goods.go*

统一链路 + 三个路由的中间件链

活动	路由	中间件链
优惠券	`POST /coupon/claim`	(仅全局 TokenBucket)
秒杀	`POST /skill_product/skill`	SlidingWindow{seckill, 3/s, ByUser}
抢红包	`POST /redpacket/claim`	SlidingWindow{redpacket, 3/s, ByUser} + Idempotency

routes/router.go:22 全局桶；*:72 coupon*；*:79–84* 红包；*:136–143* 秒杀

优惠券：库存 + 限领二维原子

coupon Lua 脚本（业务画像 + 10 行实现）

`coupon:batch:{id}:stock` 整型库存；`coupon:user:{uid}:batch:{id}` 单人计数 TTL 24h。
Lua 必须一锤完成：库存 > 0？ 单人 < `PerUser`？ 两道闸过了才 DECR + INCR。

Listing 1: *repository/cache/coupon.go:30–44 claimScript*

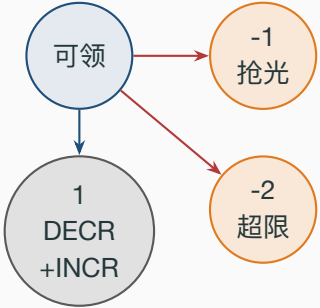
```lua
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil or stock <= 0 then
   return -1 -- 已抢光
end
local owned = tonumber(redis.call('GET', KEYS[2]))
if owned == nil then owned = 0 end
if owned >= tonumber(ARGV[1]) then
   return -2 -- 超出单人上限
end
redis.call('DECR', KEYS[1])
redis.call('INCR', KEYS[2])
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
return 1
```

Lua 脚本逐行解读

- **L1–L4**：读库存，**nil 当 0 处理**。返回 `-1` 给业务层翻译成”已抢光”。
- **L5–L9**：读单人计数。第一次领该券时 key 不存在，Lua 里 `nil` 强转 0；命中上限直接返回 `-2`。
- **L10–L11**：DECR / INCR 在 Lua 里是同一个事务原子单位，不会出现”扣了库存但没记上单人”。
- **L12**：每次领到都续命 24h ——防止 key 在活动跨天时被 evict 导致重新可领。
- 三个 return 码业务层映射成两种业务错误（见 `ClaimCouponAtomic`）。

repository/cache/coupon.go:57–75 错误码翻译

二维原子图 + REPORT 数字



	Redis Lua	DB 锁
500 并发抢 100 张	零超发	零超发
吞吐 RPS	51,362	50,142
avg 延迟	1.24ms	1.29ms
max 延迟	**136ms**	**453ms**

两

道闸都要原子：先 DECR 再判单人，已超限的请求会留” 幽灵库存”。
吞吐拉平，但 Lua vs FOR UPDATE **尾延迟差 3.3 倍**。*repository/cache/coupon.go:30–44*；*stressTest/REPORT.md* 链路压测表第 *5–6* 行

coupon 的盲区：Lua 防超发但不防黑产

- Lua 看的是 `userId+batchId`：一个账号最多领 `PerUser` 张。
- 但羊毛党的玩法是**一万个账号每号领一张**，Lua 看不出来。
- 现状：router 上只有全局 TokenBucket 100 RPS / IP，单 IP 可以慢慢刷。
- 改进建议：追加 `SlidingWindow{Scope:"coupon", Limit:5, Window:1m, ByUser:true}`。
  - 真用户 1min 领 5 张够用。
  - 万号脚本：要么换号慢，要么被网关 TokenBucket 截掉。

routes/router.go:72 coupon/claim 路由（无单用户限频）

秒杀：滑动窗口 + 单独商品表

秒杀和” 普通下单” 的根本差异

- 库存独立：`model.SkillProduct` 与 `Product` 分表，不污染主商品池。
- 限流前置：`/skill_product/skill` 强制挂 SlidingWindow，单用户 1s 最多 3 次。
- 没有熔断：秒杀打开熔断 = ” 所有人都买不到”，业务等价于” 活动作废”，比限流损失更大。
- 没有 outbox：当前实现**只读缓存**，扣库存的真实路径留在 deck 11（库存业务）。

service/skill_goods.go:122–134 SkillProduct；routes/router.go:136–143

SlidingWindow Lua 怎么算窗口 + 阈值取 3 的依据

member = 请求 id，score = 毫秒时间戳。Lua 先 `ZREMRANGEBYSCORE` 清窗外旧记录，再 `ZCARD` 判定。

阈值 = 3：真人 300–500ms 一次，3 留 50% 余量；取 10 形同没限，取 1 网络抖一下客诉爆。**比真人快、比脚本慢**，没有第二候选。*repository/cache/ratelimit.go:18–34*；*routes/router.go:136–143*

REPORT 数字：3/s/用户误差 2.2%

场景：30 VU × 15s 打 `/skill_product/skill`，单 token，配置 `Limit=3 Window=1s ByUser=true`。

指标	实测	业务期望
通过请求数	46	15s × 3 = 45
被限流数	781,624	黑产应当被全部拦下
误差	2.2%	< 5% 视为合格

”通过 46 次” 约等于” 真用户最快也只能秒到 45 次/15s”，业务侧合理。

被截 78 万次都是同一 token，正是黑产的标准画像。*stressTest/REPORT.md §5* 限流中间件

抢红包：LPOP 一次取一份

为什么红包不能用 DECR：发包时预拆 + 二倍均值法

业务约束：总额严格守恒（100 元发出去必须正好 100 元），每份金额不同 → DECR 拿不回金额，必须 LPOP 出队拿” 那一份”。**发包时一次性拆好**，写入 Redis LIST。

Listing 2: *repository/cache/redpacket.go:107–141 SplitRedPacket*

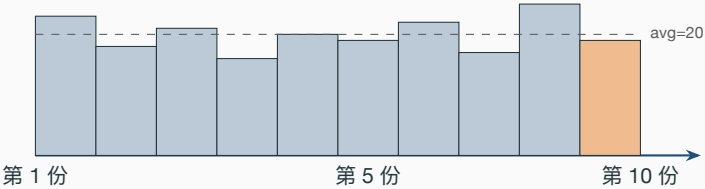
```go
out := make([]int64, count)
remain := total
for i := 0; i < count-1; i++ {
   left := int64(count - i)
   avg := remain / left * 2 // 二倍均值
   if avg < 2 { avg = 2 }
   maxThisRound := remain - int64(count-1-i) // 留至少 1 分给后面每份
   hi := avg - 1
   if hi > maxThisRound { hi = maxThisRound }
   if hi < 1 { hi = 1 }
   amt := splitRand.r.Int63n(hi) + 1 // [1, hi]
   out[i] = amt
   remain -= amt
}
out[count-1] = remain // 最后一份兜底
```

二倍均值逐行解读

- **avg = remain/left * 2**：剩余金额除以剩余份数 × 2，作为”本次最大可分到金额”的基础。
- **maxThisRound**：留给后面每份至少 1 分（避免最后几份被迫为 0）。
- **Int63n(hi)+1**：在 `[1, hi]` 闭区间随机，保证每份至少 1 分。
- **out[count-1] = remain**：最后一份直接吃剩，确保总和守恒。
- **Shuffle**：不打散的话，越靠前抢到的期望金额越大——用户会推断”第一个抢的最大”。
- **splitRand 进程内独占 rand**：`math/rand` 全局源加锁，热点路径并发不友好。

repository/cache/redpacket.go:92–142

二倍均值法分布示意（200 元 / 10 份）



每份在 `[1, 2*avg-1]` 之间

随机，期望约等于 avg，方差由 hi 上界控制。
末位（橙色）是 `remain` 兜底，shuffle 后位置随机。*repository/cache/redpacket.go:113–141*

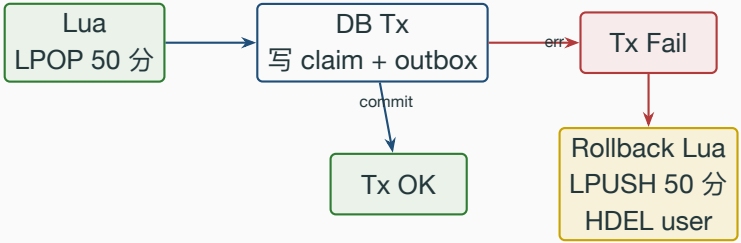
抢红包 Lua 脚本（8 行）

Listing 3: *repository/cache/redpacket.go:52–63 claimRedPacketScript*

```lua
if redis.call('HEXISTS', KEYS[2], ARGV[1]) == 1 then
   return -2 -- 已领过
end
local amt = redis.call('LPOP', KEYS[1])
if not amt then
   return -1 -- 已抢完
end
redis.call('HSET', KEYS[2], ARGV[1], amt)
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
return tonumber(amt) -- 抢到金额
```

HEXISTS 前置防同用户多抢；**LPOP 而非 BLPOP** 抢完即 `-1`（业务码 RP-EMPTY）不阻塞连接；**return amt** 省一次 `HGET` 回查。与 coupon 对比：coupon 是对称扣减（计数器），红包是资源出队（队列）。

抢红包 Saga：Lua 成功但 DB 失败怎么办



把已 LPOP 的金额 `LPUSH` 回队头、同时撤销 `claimed` 标记，用户下次再抢可以正常领。

对应 `rollbackRedPacketClaimScript`：两个动作放在一个 Lua 原子单元。

repository/cache/redpacket.go:86–90 rollback；service/redpacket.go:155–162 调用

三个 Lua 在做什么：维度对照 + 选型逻辑

维度	优惠券	秒杀（限流）	抢红包
核心命令	DECR + INCR	ZADD + ZCARD	LPOP + HSET
数据结构	STRING × 2	ZSet	LIST + HASH
原子单元	库存 + 单人计数	时间窗口集合	出队 + 去重标记
预热成本	一次 SET 库存	无（按需建窗口）	**发包时全量预拆**
TTL 策略	用户标记 24h	窗口大小 1s	claimed hash > 红包过期
回滚	INCR + DECR	无（不可逆）	LPUSH + HDEL

反过来用就不行：红包用 DECR → 并发算金额不守恒；优惠券用 LPOP → 浪费；限流用 DECR → 固定窗口边界 2× 突刺。

业务语义决定结构选型：金额异质 → 队列；同质 → 计数器；时间窗口 → ZSet。

repository/cache/coupon.go, ratelimit.go, redpacket.go 三处实现

三段 Lua 的共同设计纪律

1. **失败也要原子**：判断条件不通过时**什么都不写**，直接 return 错误码（-1 / -2）。
2. **回滚也是 Lua**：DB 失败时的补偿动作要走**同一种原子机制**，不能 Go 侧两次 Redis 调用。
3. **TTL 永远要设**：尤其是抢/领标记类 key，否则用户停留态 key 会无限堆积。
4. **返回值留语义**：用 `>0` / `0` / 负数区分成功金额、抢完、已抢过、超限，业务层 switch 即可。
5. **进程内 rand**：发包拆分时**不要用 math/rand 全局**，热点路径并发争锁。

repository/cache/redpacket.go:92–96 splitRand 局部源；*coupon.go:30–44* 错误码语义

防黑产：分层与现状

为什么营销是黑产首选 + 四层防线

黑产为何盯营销：真金白银（券能转卖、红包能提现）+ 规则透明（必须对外通告）+ 多账号便宜（手机号能买）。本质是**把作弊单价拉到 > 收益**。

层级	工具	gomall 现状	备注
L1 网络	Anti-DDoS / CDN 黑名单	未覆盖	依赖云厂商
L2 网关	IP TokenBucket	已接	router.go:22
L3 业务	用户 SlidingWindow	部分	秒杀 + 红包，coupon 缺
L4 风控	设备指纹 / 行为模型	路线图	接外部风控

每多一层，黑产成本翻倍：

刷券既要换 IP（L2）、又要换号（L3）、又要换设备指纹（L4），单券薅羊毛就不划算了。gomall 站住 L2 + L3。*routes/router.go:22* 全局 *IP* 桶；*:79–84, :136–143* 用户滑动窗口

IP TokenBucket vs 用户 SlidingWindow 选哪个

维度	TokenBucket (L2)	SlidingWindow (L3)
key 维度	ClientIP	userID
精度	桶内突发可过冲	严格窗口
绕过成本	换 IP / 走代理	换账号
位置	全局，所有请求	营销 / 高敏接口

TokenBucket 挡的是” 机器自动化”，SlidingWindow 挡的是” 账号刷子”。

两层一起上，黑产**必须同时凑齐 IP 池 + 账号池**才能突破，门槛指数级抬高。

middleware/ratelimit.go TokenBucket vs SlidingWindow

客服话术：业务码到提示文案

状态码	含义	客服话术
70001	限流命中（SlidingWindow）	” 操作过于频繁，请稍后再试”
70002	熔断打开（CB-OPEN）	” 支付服务暂忙，1 分钟后”
60002	幂等命中	” 您的操作已受理”
50001	库存不足	” 已售罄” / RP-EMPTY
-2 (coupon)	单人超限	” 每人限领 N 张”

业务侧约

定：70001 不是失败，是” 稍后重试就行”；RP-EMPTY 是终态，告诉用户红包已被抢完。客服话术必须**区分**两种” 稍后再试”，否则用户白等。*pkg/e/code.go:53–58* 业务码定义

三个活动的 SLO 与各角色视图

活动	可用率	p99	备注
优惠券领取	99.9%	< 200ms	大促日双倍预热库存
秒杀下单	99%	< 200ms	反黑产强需求，可接受偶发误伤
抢红包	99.9%	< 300ms	资金敏感，Saga 必须兜底

角色	关心指标	数据来源
运营	用券率 / 红包领完率	coupon batch + finished 数
风控	70001 / 60002 占比	业务码统计
客服	投诉单 □ 业务码反查	status 字段
SRE	Lua RTT p99 / Redis CPU	监控大盘
财务	红包总额守恒	DB sum vs outbox

秒杀可用率最低——” 全场抢光” 是营销目标，5xx 短抖动不算事故。红包反过来：抢到没入账 = 投诉。70001 比例上升，运营看的是” 参与热度”，风控看的是” 被刷”。对照 *deck 10 P0–P3* 等级表；*pkg/e/code.go*

拼团 + 满减：营销玩法的两个常见扩展

拼团把”个体优惠”做成”社交裂变”

- 拼团的业务公式：**N 人成团 = 单价 × 0.6**（举例），凑不齐就退款；凑齐了所有人都享受团购价。
- 和优惠券、秒杀的最大差别：**结果不是抢的瞬间确定的**，而是**在时间窗结束后才知道**。
- 三个关键参数决定团是否能形成：**N**（成团人数 2/3/5）、**T**（时间窗 24h/48h）、**发起人特权**（能否提前结团 / 自动开二期）。
- 资金路径：开团瞬间**付全款**（不是定金），凑齐瞬间扣的就是团购价；流团了走批量退款。
- 状态机：`Open` → `Joining` → `Full` → `Success`；或 `Joining` → `Expired` → `Refunded`。和订单 `auto_cancel_at` 一个套路：**cron 兜底 + RMQ 延时双保险**。
- 并发要点：Redis `HSET groupon:{id}:members userId 1` + `HLEN` 判定人数。Lua 一锤 HEXISTS→HLEN→HSET，形状**和 coupon claim 几乎一样**，只是 INCR 换成 HSET。

路线图：*service/groupon.go*；参照 *repository/cache/coupon.go:30–44* + *service/order_task.go* cron

满减 vs 折扣券：先算满减再算券

业务规则：Type=1 满减按订单金额自动触发，Type=2 折扣用户领取后勾选使用。叠加时**先满减再券**——反过来用户感觉被坑（券打掉单价后凑不够档位）。多档满减按阈值倒序匹配，命中第一档即返回。

Listing 4: 阶梯满减按阈值倒序匹配 + 先满减再券

```go
type FullReduceTier struct {
   Threshold int64 // 满 100 元
   Amount int64 // 减 10 元
}

// Tiers 必须按 Threshold 降序排列（DB 查出时 ORDER BY threshold DESC）
func ApplyFullReduce(tiers []FullReduceTier, amount int64) int64 {
   for _, t := range tiers {
      if amount >= t.Threshold {
         return t.Amount
      }
   }
   return 0
}
```

满减的”凑单”行为 + 三个业务陷阱

- 用户为凑满 200 减 30 硬塞一件 30 元小商品——**客单价提升的关键武器**。
- **陷阱 1**：**凑单退货导致主订单不满档**。业内主流：**部分退款时按比例摊销满减**（退款金额 = 原金额 × (1 - 满减率)），避免薅羊毛”退一件保留满减”。
- **陷阱 2**：**档位过密**（满 100 减 10、满 110 减 12、满 120 减 14）——用户决策疲劳反而不下单。建议 **3 档以内 + 跨度明显**。
- **陷阱 3**：**多店铺归集口径**。是按订单总价还是单店分摊？店铺承担成本时一定是后者。
- **退款规则**：满减券一般不退、会员券一般退——由**券类型**决定，对照 `model.UserCoupon.Status`。

对照 *deck 09* 订单状态机 *Refunding / Refunded* 节点：退款时需要重算满减摊销

排行榜 + 直播打赏：高频写场景的两种姿态

排行榜：ZINCRBY 累加销量 + 顺手裁尾控内存

双 11 战报、店铺销量榜、达人榜——排行榜是**从众心理的杠杆**。营销价值：Top 3 曝光**比首页推荐位还值钱**，带来 5–10x 自然流量。

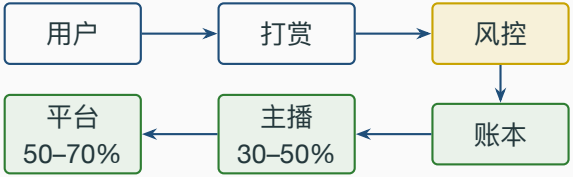
Listing 5: ZINCRBY 累加销量；超过 N=1000 时裁掉尾部

```lua
-- KEYS[1] = leaderboard:day:20260516
-- ARGV[1] = productId ARGV[2] = delta
-- ARGV[3] = maxKeep ARGV[4] = ttlSec
redis.call('ZINCRBY', KEYS[1], tonumber(ARGV[2]), ARGV[1])
local n = redis.call('ZCARD', KEYS[1])
if n > tonumber(ARGV[3]) then
   redis.call('ZREMRANGEBYRANK', KEYS[1], 0, n - tonumber(ARGV[3]) - 1)
end
redis.call('EXPIRE', KEYS[1], tonumber(ARGV[4]))
return redis.call('ZSCORE', KEYS[1], ARGV[1])
```

`ZINCRBY` 原子，比 GET-SET 安全；`ZREMRANGEBYRANK` 顺手裁尾挡**冷数据膨胀**。**实时榜的" 实时" 是错觉**：真做秒级回流会撕掉 Redis，现实是 1min 批量。日榜 `lb:day:YYYYMMDD`/TTL 48h；周榜凌晨 cron `ZUNIONSTORE` 7 个日 key。

路线图：*repository/cache/leaderboard.go*；*ZSet 模板对照 repository/cache/ratelimit.go:18–34*

直播打赏：高频小额 + 主播分成 + 风控两条线



业务画像：一晚上几万次打赏，单次 1–500 元，**比下单还高频 100 倍**；和抢红包链路骨架**完全一致**：钱包扣账 + outbox + 异步聚合。**差异**：礼物**不可撤销**（飘屏发了不能收回），**不需要 Saga**；只要**先扣钱再发特效**的顺序。

风控规则：单 IP 1min 内打赏 > 1000 次、单账号充值后 5min 内全打赏给同一主播 = 大概率洗钱 / 刷榜。

结算：主播 T+1 提现给风控冷却 + 退款窗口；未实名 + 单日 > 500 强制弹窗” 家长确认”。

业务路线图：*service/gift.go* 骨架可复用 *service/redpacket.go:42–103 outbox* 通道

红包深挖 + 风控决策树

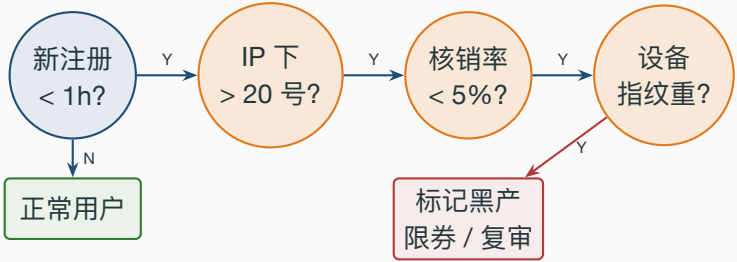
群红包 / 平台红包 / 拼手气 / 普通红包业务模型

维度	群红包	平台红包	拼手气	普通红包
资金来源	个人钱包	营销预算	个人钱包	个人钱包
拆分方式	拼手气 / 平均	拼手气	二倍均值法	等额
领取门槛	群成员	全平台	群成员	指定用户
合规风险	低	高（红包券）	低	极低
gomall 现状	路线图	路线图	**已实现**	退化版（n=1）

- **平台红包**本质是营销返券，监管口径接近”消费券”，要**核销而非提现**——避免被认定为变相返利。
- **营销预算池**：`marketing_budget{campaignId}` 与 `user_wallet{userId}` 二级账户隔离，预算有日限额、写穿就触发熔断（前端表现为”红包已抢光”）。
- **拼手气**才需要二倍均值法；**等额**更简单（total/count）。**群红包**多一层”群成员校验”。

当前 *service/redpacket.go* 是”拼手气”模型；平台红包需扩展 *model.RedPacket* 加 *campaignID*

黑产识别决策树：风控四问级联



黑产手法：批量注册（接码 8 分/号）、脚本秒抢（1000 RPS）、设备模拟、券转卖。

早期信号：新注册 < 1h 就领大额券；单 IP > 50 号都领过同批次；**核销率 < 5%**（拿券不下单 = 倒卖）。

级联决策：任一问 N 放行；4 问全 Y 才打标。**灰名单**降级（额度小 / 限领 1），**黑名单**拒发；7 日内自动出灰名单留误伤余地。路线图：*middleware/risk_control.go*；模板参照 *TokenBucket*

大促经验 + 优惠生命周期 + 容量规划

双 11 / 618 / 百亿补贴：gomall 能力对照

大促	流量画像	库存机制	黑产对抗
天猫双 11 0 点 京东 618 拼多多百亿补贴	30s 内 50 万 RPS 持续半月，每日有峰 全年常态	预扣 + 分布式锁 同上 + 区域库存 每日小批次 + 限购	设备指纹 + 风控雷达 实名 + 历史行为 重在凑单识别

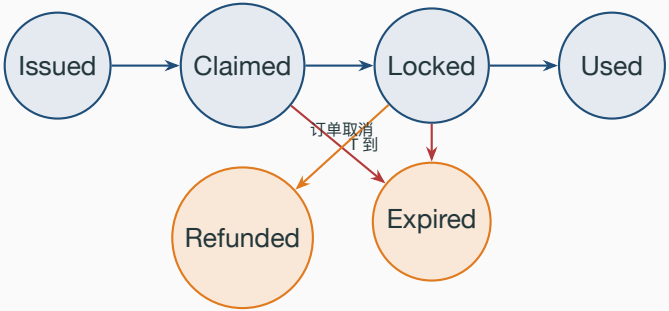
共性：预热阶段让用户领券 / 加购 / 关注店铺——0 点只做” 结算” 不做” 决策”，把决策从瞬时挪到长尾。

能力维度	gomall 现状	双 11 级要求
路由层吞吐	64K RPS (单机)	50 万 + RPS
单接口尾延迟	p99 < 5ms / max 136ms	p99 < 50ms
熔断 / 限流	TokenBucket + SlidingWindow	同 + 风控雷达
Redis 容量	单机	集群 (64+ 分片)

结论：结构对、规模差一个数量级。

单机 □ 集群是堆机器问题；缺的是**风控雷达 + 区域库存调度**。*stressTest/REPORT.md* 链路压测表第 *1–7* 行；对照 *deck 07* 库存 + *deck 10* 限流熔断

优惠券生命周期 + 核销时机三选一



Locked 是关键中间态：下单时预占券（避免一券下两单），订单取消回 Claimed。**Expired** 不可复活，财务侧作为营销 ROI 的**沉没成本**。

- **下单立扣**（gomall + 淘京）：用户预期" 用券价" 是结算瞬间事实；订单关单要回滚券。
- **支付立扣**：付款才扣，不用退；但用户看到的优惠和最终扣的可能不一致。
- **完成立扣**（收货）：理论最严谨；财务对账困难（核销时间太晚）。

A/B 测试 + 转化漏斗 + 容量规划

A/B 公式：userId 哈希% 100 分桶，z-test 双比例 `p < 0.05` 才认。**分桶维度必须用 userId 而非 sessionId**——同用户多次访问要进同桶。

节点	A 桶 (10 元)	B 桶 (15 元)	差距
曝光	100,000	100,000	—
下单	1,260	2,160	B +71%
核销 GMV	320,000 元	540,000 元	B +69%
券成本	12,600 元	32,400 元	B +157%
净 GMV	**307,400 元**	**507,600 元**	**B +65%**

容量规划：春晚级红包 1 千万用户瞬时打 50% = 500 万 RPS。单 Lua 在快路径 20–40K QPS，需 **50–150 个 Redis 分片**并联。分片 key：`redpacket:{id}:amounts:{shard}`，按 userId hash 选分片；空分片靠对账兜底回收。路线图：*middleware/experiment.go*；对照 *repository/cache/redpacket.go:34–43* 单 *list* 实现

面试 Q&A（一）：原子性与回滚

Q1：为什么 coupon 用 DECR，红包用 LPOP?

A：券同质，整型计数器即可；红包每份金额不同，必须出队拿”那一份”。

代码：`repository/cache/coupon.go:30--44`、`redpacket.go:52--63`。

Q2：coupon Lua 里”先查库存再查单人”能不能换顺序?

A：可以换，但语义有别：先查单人会先消耗 user key 的读，库存爆满时反而更慢。当前顺序对”已抢光”这种最常见的失败短路更划算。

代码：`repository/cache/coupon.go:31--39`。

Q3：红包 Lua 已经 LPOP 了，DB 落库失败怎么办?

A：Saga 回滚——跑 `rollbackRedPacketClaimScript`，`LPUSH` 金额回队头 + `HDEL` 撤销 claimed 标记，用户下次能再抢。

代码：`repository/cache/redpacket.go:86--90`、`service/redpacket.go:155--162`。

面试 Q&A（二）：风控与限流

Q4：3/s/用户这个数从哪来?

A：真人点击 300–500ms 一次，3 留 50% 余量。压测 46/45 误差 2.2%。

代码：`routes/router.go:145--152`；压测 `stressTest/REPORT.md §5`。

Q5：羊毛党用一万个账号刷 coupon，gomall 现在能挡住吗?

A：挡不住。需要追加 `SlidingWindow{coupon, 5/min, ByUser}`，再叠加风控四问决策树。当前只能靠全局 TokenBucket 100 RPS/IP 拖速度。

代码：`routes/router.go:22, 70--73`。

Q6：秒杀为什么不挂 Idempotency?

A：秒杀重复点击的语义就是” 再试一次”，幂等命中反而误导用户。限流截掉就够了。

代码：`routes/router.go:145--152`（无 Idempotency）。

面试 Q&A（三）：拼团与满减

Q7：拼团时间窗到了部分用户没付怎么办？

A：拼团强制开团瞬间付全款，不存在”没付”；T 到了凑不齐就批量退款。状态机 Joining → Expired → Refunded，由 cron + RMQ 延时双兜底。

代码：路线图 `service/groupon.go`，参照 `service/order_task.go` cron。

Q8：满减券和折扣券一起用，先算哪个？

A：先算满减再算券。先扣大头，券作用在剩余金额上；反过来用户感觉被坑（券打掉单价后凑不齐档位）。

代码：路线图 `service/promotion.go`，已有 `model.CouponBatch.Type/Threshold/Amount` 字段支撑。

Q9：满减 + 部分退款怎么处理？

A：按比例摊销满减——退款金额 = 原金额 × (1 - 满减率)，避免”退一件保留满减”的薅羊毛。

代码：路线图，对照 deck 09 Refunding/Refunded 节点。

面试 Q&A（四）：排行榜 / 直播 / A/B

Q10：排行榜怎么防止某些商品永远霸榜导致内存膨胀？

A：`ZADD` 后顺手 `ZREMRANGEBYRANK` 裁掉最低分的，最多保留 Top 1000；TTL 控制冷榜过期。

代码：路线图 `repository/cache/leaderboard.go`，Lua 模板与 `ratelimit.go:18--34` 一致。

Q11：直播打赏要不要做 Saga 回滚？

A：不要。礼物特效一旦显示不可撤销，业务上等价于”用户接受了”。只需保证**先扣钱再发特效**的顺序，DB 失败时直接报错给用户，不回滚特效（特效本身还没发）。

代码：业务路线图，骨架参照 `service/redpacket.go:42--103`。

Q12：A/B 测试的分桶为什么不能用 sessionId？

A：同一用户多次访问要进同一桶，否则用户在不同访问看到的优惠不一致，转化率失真。userId 哈希才稳定。

代码：业务路线图 `middleware/experiment.go`。

代码位置一览（一）：当前实现

coupon Lua + 错误码 *repository/cache/coupon.go:30–75*

coupon 业务层 *service/coupon.go:65–99 Claim*

coupon 库存预热 *service/coupon.go:30–57 CreateBatch*

红包拆分算法 *repository/cache/redpacket.go:102–142 SplitRedPacket*

红包 prepare Lua *repository/cache/redpacket.go:34–43*

红包 claim Lua *repository/cache/redpacket.go:52–63*

红包 Saga 回滚 *repository/cache/redpacket.go:86–90 + service/redpacket.go:155–162*

红包过期回收 *repository/cache/redpacket.go:69–77 ReleaseRedPacketLeft*

秒杀业务 *service/skill_goods.go:122–134*

秒杀路由 + 滑动窗口 *routes/router.go:145–152*

红包路由 + 滑动窗口 + 幂等 *routes/router.go:79–84*

SlidingWindow Lua *repository/cache/ratelimit.go:18–34*

业务码 *pkg/e/code.go:53–58*

代码位置一览（二）：路线图与对照 deck

拼团骨架	路线图 *service/groupon.go*；参照 *service/order_task.go cron* 兜底
满减引擎	路线图 *service/promotion.go*；字段已在 *model.CouponBatch*
排行榜	路线图 *repository/cache/leaderboard.go*；*ZSet* 模板参考 *ratelimit.go*
直播礼物	路线图 *service/gift.go*；*outbox* 通道参考 *service/redpacket.go:78–90*
平台红包	路线图：扩展 *model.RedPacket* 加 *campaignID* 字段
风控决策树	路线图 *middleware/risk_control.go*；模板参考 *TokenBucket*
A/B 中间件	路线图 *middleware/experiment.go*；模板参考 *SlidingWindow*

配套技术 deck：05-ratelimit / 07-inventory / 09-order-lifecycle / 10-traffic-governance / 11-consistency / 12-merchant-ops